DSA5208: Scalable Distributed Computing for Data Science

Lecture 8: Message Passing Interface (MPI)

Anthony Chen

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

MPI: Message Passing Interface

As previously mentioned, MPI is the standard in the parallel computing world for parallelization across pools of memory.

- MPI is highly optimized and can be competitive with OpenMP for parallelization within a node
- Code is frequently implemented as MPI+X, where MPI is used for parallelization across nodes and another framework is used for intra-node parallelism
- MPI is highly portable
- MPI continues to develop even now

The discussion of MPI in class will focus primarily on the underlying concepts. Many MPI tutorials exist online for more information on coding with MPI.

Implementations of MPI

As previously mentioned, MPI is a standard and not an implementation. Several implementations exist, including:

- MPICH
- Open MPI (not to be confused with OpenMP)
- Intel OneAPI
- HPE

To check if you have MPI installed, you can run `$which mpicxx`

The MPI Programming Model

An independent MPI process is called a rank. MPI ranks can interact in several ways:

- Two MPI ranks can engage in a point to point communication
- A group of MPI ranks belonging to an MPI communicator can engage in a collective communication
- A single MPI rank can engage in a remote memory access communication

Aside from these interactions, MPI ranks act independently.

Sending and Receiving Messages

The main point to point communications in MPI are `MPI_Send` and `MPI_Recv`.

```
int MPI_Send(const void *sendbuf, int count, MPI_Datatype datatype, int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *recvbuf, int count, MPI_Datatype datatype, int source, int tag,
             MPI_Comm comm, MPI_Status *status)
```

MPI Rank `source` sends an array of length `count` of data type `datatype`, located at `*sendbuf` to MPI rank `dest`, which stores the data at `*recvbuf`.

Blocking

The default `MPI_Send` and `MPI_Recv` are blocking communications, which means that they do not return until they are complete. This can pose several problems:

- MPI ranks may spend a lot of time waiting for a matching send or receive to be posted
- Deadlock may occur, where no process can proceed because it is waiting for another process

Both of these are detrimental to parallel efficiency.

Asynchronous Communication

To remedy issues around blocking, `MPI_Send` and `MPI_Recv` have several variants:

- `MPI_Ssend`: returns when the destination rank starts to receive the message
- `MPI_Bsend`: posts a send. If a matching receive is available, the message is passed. If not, the message is first copied into another buffer
- `MPI_Rsend`: a send, assumes a matching receive has already been posted
- `MPI_Isend`: returns immediately, regardless of receiver status. One has to be careful about not modifying the data before the send is complete

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

Collective Communications

Collective communications occur between all ranks of a MPI communicator. There are several main collective communications:

- Broadcast: MPI send, but every MPI rank receives the data
- Scatter: when an array is scattered, each MPI rank receives a subset of the array
- Gather: receives data from every MPI rank
- Prefix scan: $[0, 1, 2, 3, 4, 5] \rightarrow [0, 1, 3, 6, 10, 16]$
- Reduce: combines data from every MPI rank in some way

MPI Broadcast

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm)
```

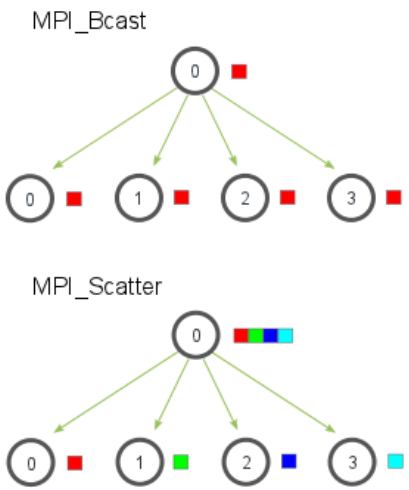
Sends some data, `buffer`, of size `count` and type `datatype` from MPI rank `root` to every other MPI rank in `comm`.

- Widely useful
- Commonly used after performing serial initialization
- More efficient than calling `MPI_Send` a bunch of times

MPI Scatter

```
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
        void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
        MPI_Comm comm)
```

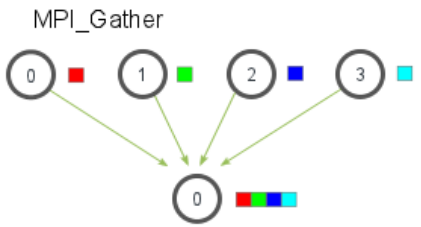
The `root` sends data from `sendbuf` of length `sendcount` and type `sendtype` to every other MPI rank in `comm`, which receives the dataof size `recvcount` and type `recvtype` in `recvbuf`. Not the same as a broadcast:



MPI Gather

```
int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
        void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
        MPI_Comm comm)
```

Opposite of `MPI_Scatter`. The `root` receives data from every other MPI rank in the communicator.

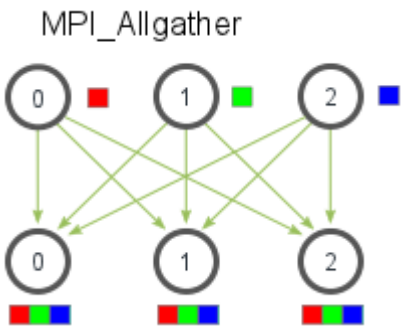


There are variants called `MPI_Scatterv` and `MPI_Gatherv` where data being scattered and gathered can vary by size among MPI ranks.

MPI Allgather

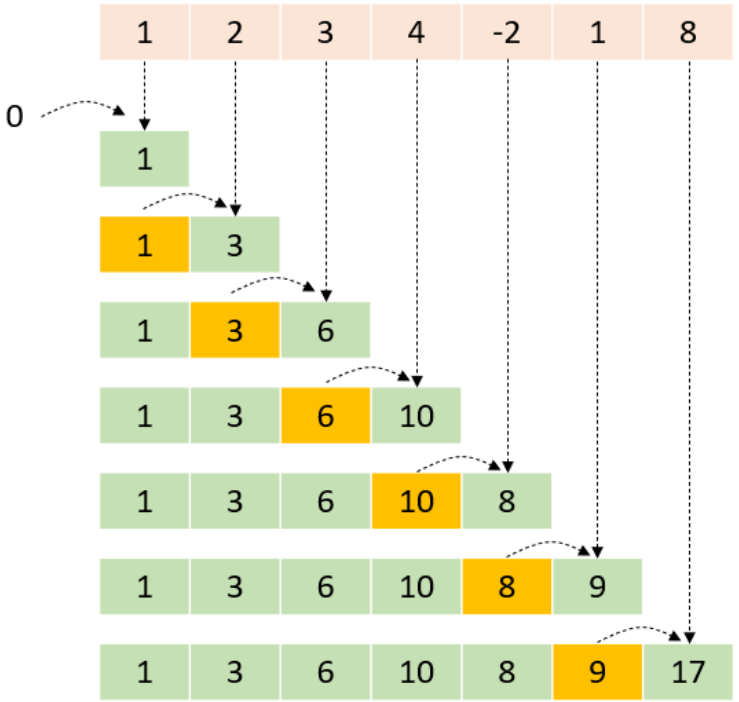
```
int MPI_Allgather(const void *sendbuf, int  sendcount, MPI_Datatype sendtype,
        void *recvbuf, int recvcount,  MPI_Datatype recvtype,
        MPI_Comm comm)
```

Another variant on `MPI_Gather`. Instead of the gather result being on a single MPI rank, every MPI rank in the communicator gets a full set of data.



There is also a variant, `MPI_Allgatherv`. There is no `MPI_Allscatter`.

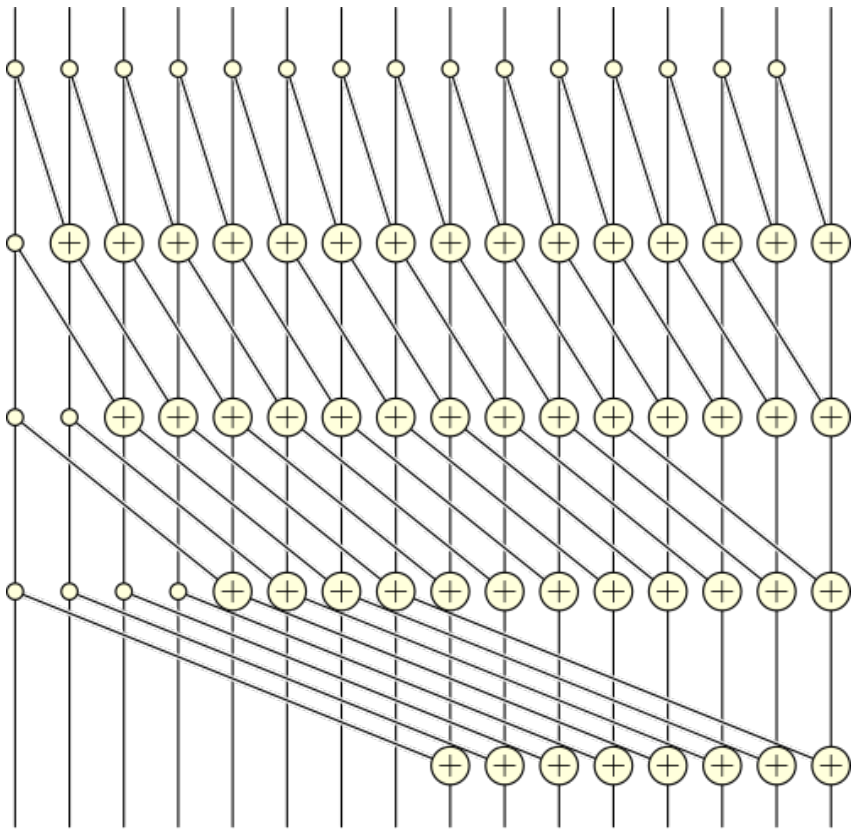
Scans



Also known as a prefix sum or prefix scan. Obviously $O(n)$ in serial. How do you parallelize it?

A parallel scan

One method (Hillis-Steele) is as follows:



In serial, this is $O(n \log n)$, but with n processors, this becomes $O(\log n)$.

MPI Scan

Fortunately, you don't need to worry about the implementation (unless you want to do this for your project).

```
int MPI_Scan(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype,
                MPI_Op op, MPI_Comm comm)
```

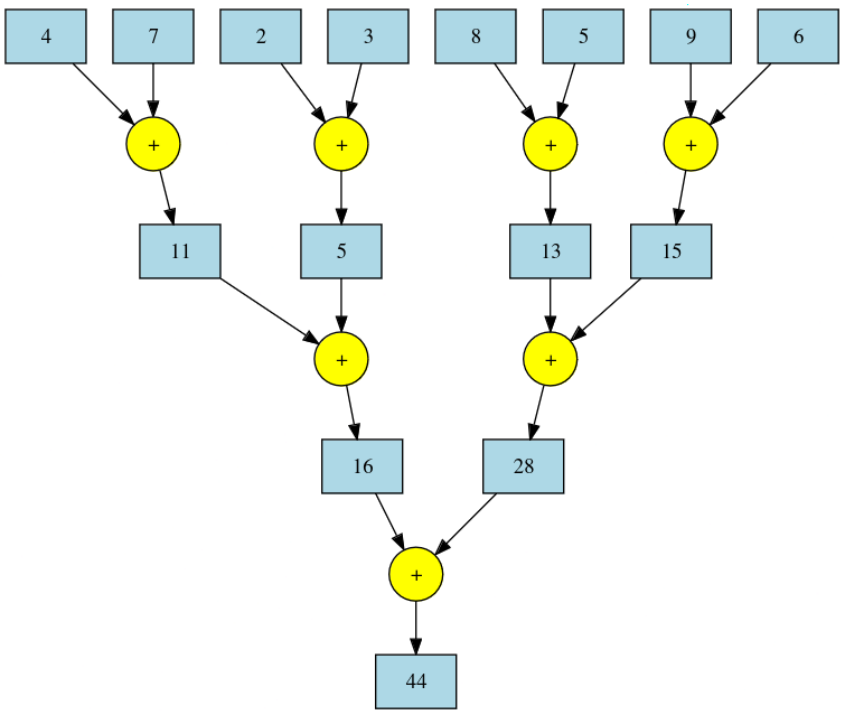
The function signature is pretty standard, except there is an additional `MPI_Op op` input. Some common operations include:

- `MPI_MAX`
- `MPI_MIN`
- `MPI_SUM`
- `MPI_PROD`

Custom operations are possible as well.

Reductions

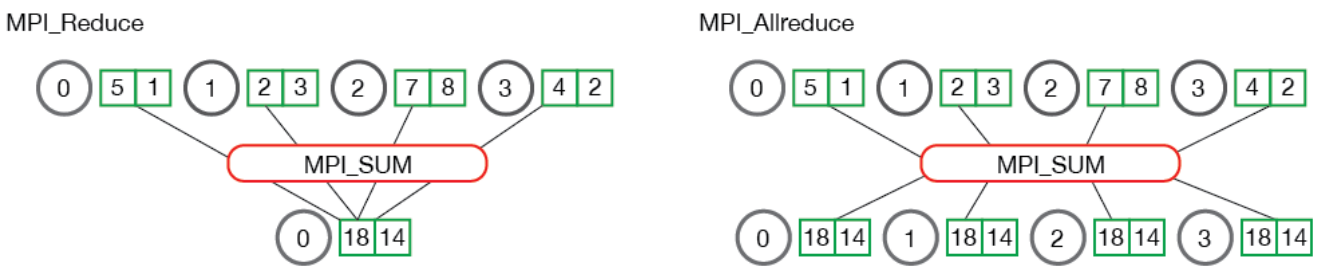
A reduction turns an array into a single number. To parallelize, a binary tree can be used:



MPI Reduce

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root,
               MPI_Comm comm)
```

As with the scan, there is a choice of operations available. There is also a variant, `MPI_Allreduce`.



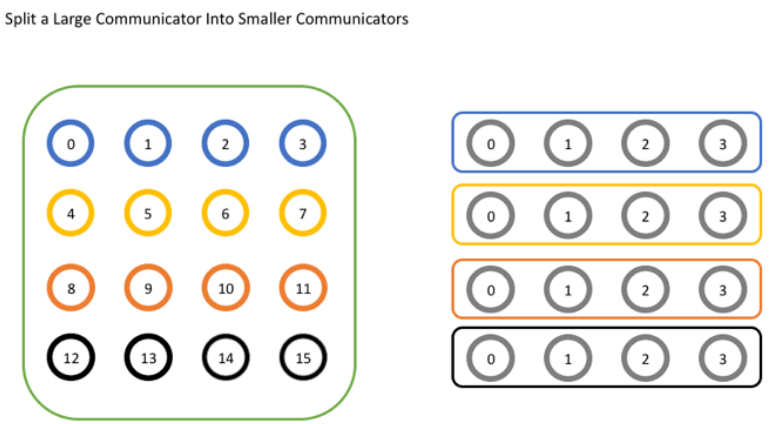
MPI Communicators

There has been this mysterious `MPI_Comm comm` floating around that has not been explained much. A MPI communicator has two components:

- An ID
- A list of MPI ranks associated with that communicator

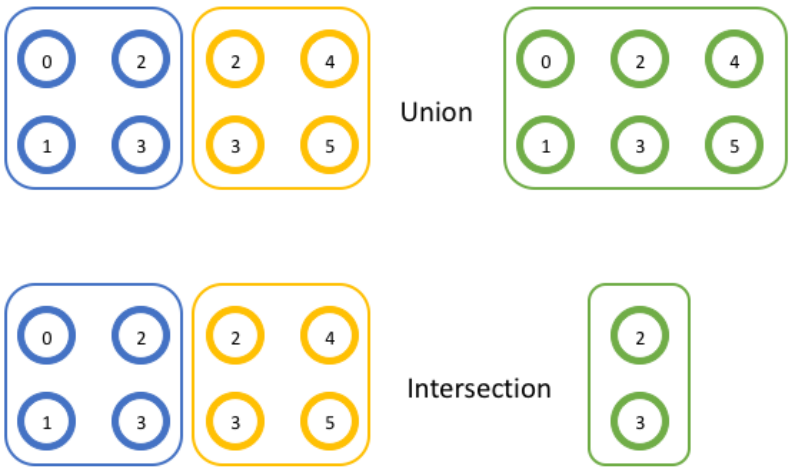
Every MPI program has at least one MPI communicator, `MPI_COMM_WORLD`. However, there are situations where you want more fine grained control over collective communications. There are two tools for this:

- Splitting an existing MPI communicator into new communicators
- MPI groups



MPI Groups

A MPI Group is like a MPI Communicator, but you can also do operations on MPI groups:



MPI communicators can be turned into MPI groups and vice versa.

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

Remote Memory Access

Remote Memory Access (RMA) is another communication paradigm in MPI.

- In contrast to message passing, no coordination between MPI ranks is needed
- Therefore, useful when synchronization is difficult
- Shares some similarities with shared memory programming
- The main communication operations are `MPI_Put`, `MPI_Get`, and `MPI_Accumulate`

MPI Windows

To put or get data from another MPI rank, some information about memory locations is needed. This is where the window comes in:

- A window is created by every MPI rank in a communicator
- A window exposes some region of a MPI rank's memory to other MPI ranks
- The size of the window is specified as a number of bytes

```
MPI_Win_create(void *base, MPI_Aint size, int disp_unit,
               MPI_Info info, MPI_Comm comm, MPI_Win *win)
```

RMA Operations

Three main operations.

```
MPI_Get(void *origin_addr, int origin_count, MPI_Datatype
        origin_datatype, int target_rank, MPI_Aint target_disp,
        int target_count, MPI_Datatype target_datatype, MPI_Win win)
```

```
MPI_Put(const void *origin_addr, int origin_count, MPI_Datatype
        origin_datatype, int target_rank, MPI_Aint target_disp,
        int target_count, MPI_Datatype target_datatype, MPI_Win win)
```

```
int MPI_Accumulate(const void *origin_addr, int origin_count,
        MPI_Datatype origin_datatype, int target_rank,
        MPI_Aint target_disp, int target_count,
        MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
```

Put is usually faster than Get. All RMA operations are nonblocking.

Synchronization

Even though RMA operations are nonblocking, synchronization is still needed.

- How can a target know when a put has finished and access the data?
- How can an origin know when a put has finished and modify the data?
- How can an origin know when an accumulate has finished and use the result?

Fencing

The simplest way of synchronizing RMA operations is with a window fence.

```
int MPI_Win_fence(int assert, MPI_Win win)
```

This will block until all RMA operations using the window are completed. However, this might not always be the most efficient method.

Passive Target Synchronization

Completion of `MPI_Get` on the origin should not require any action on the part of the target. This leads to the window lock and unlock.

```
int MPI_Win_lock(int lock_type, int rank, int assert, MPI_Win win)
```

```
int MPI_Win_unlock(int rank, MPI_Win win)
```

These are not called collectively. As a result, these are scalable and more suitable for less uniform RMA operations.

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

Floating Point Arithmetic

Computers represent real numbers as floating point numbers. IEEE 754 specifies a floating point number as consisting of three components:

- A sign, s
- An exponent, e
- A mantissa, b

Given these, the corresponding value is $(-1)^s 2^{e-e_0}(1+b)$. e_0 is a constant determining some shift, allowing for negative magnitudes as well. For example, the standard 32 bit floating point format designates 1 bit for the sign, 8 for the exponent, and 23 for the mantissa. As an example, with $s = 1$, $e = 10000001_2$, and $b = 01100000000000000000000_2$, one has $e = 129$ and $b = 0.375$. $e_0 = 2^8 = 128$, so the corresponding floating point value is $(-1)^1 e^{129-127}(1+0.375) = -5.5$.

Why is this important?

Finite precision of floating point numbers means that most numbers are not represented exactly. As a result, floating point addition is not associative.

$$(a + b) + c \neq a + (b + c)$$

Reproducibility

Because floating point addition is not associative:

- `MPI_Reduce` and `MPI_Accumulate` will likely produce different answers across different runs, even with the same input
- The output of a parallel program may vary slightly between runs
- This can make debugging challenging
- Certain applications also prize reproducibility

Solutions? A possible topic to consider for your project.

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

Gaussian Kernel Ridge Regression

Consider a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Kernel ridge regression takes the form

$$f(x) = \sum_{i=1}^{N} \alpha_i k(x, x_i)$$

where k is some kernel and α_i are the coefficients of interest, satisfying

$$\lambda \alpha_i + \sum_{j=1}^{N} \alpha_j k(x_i, x_j) = y_i$$

where λ is a parameter to help avoid overfitting. The Gaussian kernel is

$$k(x, x') = e^{-\frac{\|x - x'\|^2}{2s^2}}$$

where s is some width parameter.

How do we parallelize this?

There are two key difficulties in performing GKRR. First, we have to solve a linear system:

$$\lambda\alpha_i + \sum_{j=1}^{N} \alpha_j k(x_i, x_j) = y_i \quad \Longrightarrow \quad (K + \lambda I)\alpha = y$$

Next, in some cases, the data may be distributed, so that across n processors, each one only has a portion of $\mathcal{D}$. We can write this as

$$\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_n$$

This means that forming the K matrix requires communication.

Forming the matrix

Several possibilities for how to form the K matrix:

- Processor m sends $\mathcal{D}_m$ to processor p, processor p computes $k(\mathcal{D}_p, \mathcal{D}_m)$, iterate for $p = 0$ through $n - 1$. Each processor contains a subset of the matrix.
- Collect all the $\mathcal{D}_m$ on each rank, every processor computes the full matrix.
- Other ideas?

Of course, any of these methods of forming the K matrix have multiple implementations using MPI.

Solving the Linear System

Once K is formed, how do you then solve

$$(K + \lambda I)\alpha = A\alpha = y$$

in parallel? Various methods:

- LU factorization
- Cholesky decomposition, because the matrix is symmetric and if $\lambda > 0$, the matrix is positive definite
- Iterative solvers (conjugate gradient)

Conjugate Gradient

Conjugate gradient is an iterative solver. To solve $\mathbf{Ax} = \mathbf{b}$, start with an initial guess $\mathbf{x}_0$. Compute $\mathbf{r}_0 = \mathbf{b} - \mathbf{Ax}_0$ and $\mathbf{p}_0 = \mathbf{r}_0$. Then, while the residual $\mathbf{r}_k$ is large, iterate:

- Compute $\alpha_k = (\mathbf{r}_k^T \mathbf{r}_k)/(\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k)$
- Update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
- Update $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$
- Compute $\beta_k = (\mathbf{r}_{k+1}^T \mathbf{r}_{k+1})/(\mathbf{r}_k^T \mathbf{r}_k)$
- Update $\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$

How can you parallelize this?

Parallelizing Conjugate Gradient

Some ideas:

- The most costly operations are the matrix vector multiplications, computing $\mathbf{A}\mathbf{p}_k$ is $O(N^2)$. Break up this computation?
- Do you want to parallelize any of the other computations?
- Should each MPI rank include a full copy of $\mathbf{A}$, $\mathbf{x}_k$, $\mathbf{r}_k$, and $\mathbf{p}_k$? Limits the size of the problem
- If instead each MPI rank owns a portion of $\mathbf{A}$, $\mathbf{x}_k$, $\mathbf{r}_k$, and $\mathbf{p}_k$, then you need to do more communication

On top of these top level design decisions, there are also implementation decisions.

Even More Considerations

There are even more considerations:

- Different linear solvers can be used and might result in different ways of creating/storing the matrix
- Different linear solvers will have different operation count vs communication volume
- Each communication has different ways of being performed
- Different combinations of techniques might be optimal for different problem sizes

1 MPI

2 Collective Communications in MPI

3 Remote Memory Access in MPI

4 Floating Point Arithmetic

5 An Example Problem

6 HPC Resources

University HPC Resources

The university HPC cluster has three computers:

- Atlas: CPU only, slow, unlimited use
- Vanda: Mixed CPU and GPU, limited core-hour allocation
- Hopper: GPU nodes, paid access only

User guides available at https://nusit.nus.edu.sg/hpc/

Logging In

When connected to university wifi:

- In terminal or another command line,
  `ssh username@atlas8.nus.edu.sg`
- If on Windows, install a terminal application (MobaXterm is a common recommendation, WSL can also do this)
- Enter your password (it will be hidden)

File Management

Useful commands:

- Use the command `hpc space` to check your disk quota
- Use `ls` to list the files in the current directory
- Use `cd` to change directories
- Use `scp` to copy files to/from your personal computer

Running Code

HPC machines are split into login nodes and compute nodes. Most of your interaction is with login nodes:

- SSH into a login node
- Access your files on a login node
- Compile your code on a login node
- Submit jobs on login nodes

Only after a job has been submitted is a compute node used.

Submitting Jobs

When you submit a job, several things are specified:

- Which queue you are using
- How many resources you are using
- The executable
- Runtime limit

Use `gstat` to view the status of the cluster and the queues, and use `qstat -Qf queue_name` to view the details of a particular queue. On Vanda, you can run `hpc cpu` or `hpc gpu` for a sample job submission script as well.

Compiling Code

Compiling your code on Atlas or Vanda can be a bit involved. On Atlas:

- Load the environment `source /app1/ebenv`
- Load any other modules that you might need using `module avail` and `module load`
- Compile your code

On Vanda, the process is a bit different. The login nodes on Vanda are a virtual machine, so you need to compile your code on a compute node:

- Start an interactive job with
  `qsub -I -l select=1:ncpus=1 -l walltime=00:10:00`
- Load modules
- Compile